Learn Claude Code (中文版)

> 一个工具 + 一个循环 = 一个智能体。

本文档是 12 个章节的中文完整版，从基础智能体循环逐步构建到完整的自治智能体团队系统。

---

s01: The Agent Loop (智能体循环)

`[ s01 ] s02 > s03 > s04 > s05 > s06 | s07 > s08 > s09 > s10 > s11 > s12`

> *"One loop & Bash is all you need"* -- 一个工具 + 一个循环 = 一个智能体。

问题

语言模型能推理代码, 但碰不到真实世界 -- 不能读文件、跑测试、看报错。没有循环, 每次工具调用你都得手动把结果粘回去。你自己就是那个循环。

解决方案

```
+--------+      +-------+      +---------+
|  User  | ---> |  LLM  | ---> |  Tool   |
| prompt |      |       |      | execute |
+--------+      +---+---+      +----+----+
                    ^                |
                    |   tool_result  |
                    +----------------+
                    (loop until stop_reason != "tool_use")
```

一个退出条件控制整个流程。循环持续运行, 直到模型不再调用工具。

工作原理

1. 用户 prompt 作为第一条消息。

```
messages.append({"role": "user", "content": query})
```

1. 将消息和工具定义一起发给 LLM。

```
response = client.messages.create(
    model=MODEL, system=SYSTEM, messages=messages,
    tools=TOOLS, max_tokens=8000,
)
```

1. 追加助手响应。检查 `stop_reason` -- 如果模型没有调用工具, 结束。

```
messages.append({"role": "assistant", "content": response.cont
if response.stop_reason != "tool_use":
    return
```

1. 执行每个工具调用, 收集结果, 作为 user 消息追加。回到第 2 步。

```
results = []
for block in response.content:
    if block.type == "tool_use":
        output = run_bash(block.input["command"])
        results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": output,
        })
messages.append({"role": "user", "content": results})
```

组装为一个完整函数:

```
def agent_loop(query):
    messages = [{"role": "user", "content": query}]
    while True:
        response = client.messages.create(
            model=MODEL, system=SYSTEM, messages=messages,
            tools=TOOLS, max_tokens=8000,
        )
        messages.append({"role": "assistant", "content": respo

        if response.stop_reason != "tool_use":
            return

        results = []
        for block in response.content:
            if block.type == "tool_use":
                output = run_bash(block.input["command"])
                results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": output,
                })
        messages.append({"role": "user", "content": results})
```

不到 30 行, 这就是整个智能体。后面 11 个章节都在这个循环上叠加机制 -- 循环本身始终不变。

变更内容

组件	之前	之后
Agent loop	(无)	`while True` + stop_reason
Tools	(无)	`bash` (单一工具)
Messages	(无)	累积式消息列表
Control flow	(无)	`stop_reason != "tool_use"`

试一试

```
cd learn-claude-code
python agents/s01_agent_loop.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Create a file called hello.py that prints "Hello, World!"`
2. `List all Python files in this directory`
3. `What is the current git branch?`
4. `Create a directory called test_output and write 3 files in it`

---

s02: Tool Use (工具使用)

`s01 > [ s02 ] s03 > s04 > s05 > s06 | s07 > s08 > s09 > s10 > s11 > s12`

> *"加一个工具, 只加一个 handler"* -- 循环不用动, 新工具注册进 dispatch map 就行。

问题

只有 `bash` 时, 所有操作都走 shell。`cat` 截断不可预测, `sed` 遇到特殊字符就崩, 每次 bash 调用都是不受约束的安全面。专用工具 (`read_file` , `write_file`) 可以在工具层面做路径沙箱。

关键洞察: 加工具不需要改循环。

解决方案

```
+--------+      +-------+      +------------------+
|  User  | ---> |  LLM  | ---> | Tool Dispatch    |
```

```
| prompt |      |       |      | {                 |
+--------+      +---+---+      |   bash: run_bash |
                    ^           |   read: run_read |
                    |           |   write: run_wr  |
                    +-----------+   edit: run_edit |
                    tool_result | }                |
                                +------------------+

The dispatch map is a dict: {tool_name: handler_function}.
One lookup replaces any if/elif chain.
```

工作原理

1. 每个工具有一个处理函数。路径沙箱防止逃逸工作区。

```
def safe_path(p: str) -> Path:
    path = (WORKDIR / p).resolve()
    if not path.is_relative_to(WORKDIR):
        raise ValueError(f"Path escapes workspace: {p}")
    return path

def run_read(path: str, limit: int = None) -> str:
    text = safe_path(path).read_text()
    lines = text.splitlines()
    if limit and limit < len(lines):
        lines = lines[:limit]
    return "\n".join(lines)[:50000]
```

1. dispatch map 将工具名映射到处理函数。

```
TOOL_HANDLERS = {
    "bash":       lambda **kw: run_bash(kw["command"]),
    "read_file":  lambda **kw: run_read(kw["path"], kw.get("li
    "write_file": lambda **kw: run_write(kw["path"], kw["conte
    "edit_file":  lambda **kw: run_edit(kw["path"], kw["old_te
```

```
                                                kw["new_text"]),
}
```

1. 循环中按名称查找处理函数。循环体本身与 s01 完全一致。

```
for block in response.content:
    if block.type == "tool_use":
        handler = TOOL_HANDLERS.get(block.name)
        output = handler(**block.input) if handler \
            else f"Unknown tool: {block.name}"
        results.append({
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": output,
        })
```

加工具 = 加 handler + 加 schema。循环永远不变。

相对 s01 的变更

组件	之前 (s01)	之后 (s02)
Tools	1 (仅 bash)	4 (bash, read, write, edit)
Dispatch	硬编码 bash 调用	`TOOL_HANDLERS` 字典
路径安全	无	`safe_path()` 沙箱
Agent loop	不变	不变

试一试

```
cd learn-claude-code
python agents/s02_tool_use.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Read the file requirements.txt`
2. `Create a file called greet.py with a greet(name) function`
3. `Edit greet.py to add a docstring to the function`
4. `Read greet.py to verify the edit worked`

---

s03: TodoWrite (待办写入)

`s01 > s02 > [ s03 ] s04 > s05 > s06 | s07 > s08 > s09 > s10 > s11 > s12`

> *"没有计划的 agent 走哪算哪"* -- 先列步骤再动手, 完成率翻倍。

问题

多步任务中, 模型会丢失进度 -- 重复做过的事、跳步、跑偏。对话越长越严重: 工具结果不断填满上下文, 系统提示的影响力逐渐被稀释。一个 10 步重构可能做完 1-3 步就开始即兴发挥, 因为 4-10 步已经被挤出注意力了。

解决方案

```
+--------+      +-------+      +---------+
|  User  | ---> |  LLM  | ---> | Tools   |
| prompt |      |       |      | + todo  |
+--------+      +---+---+      +----+----+
                    ^                |
                    |   tool_result  |
                    +----------------+
                          |
              +-----------+-----------+
              | TodoManager state     |
              | [ ] task A            |
              | [>] task B  <- doing  |
```

```
               | [x] task C            |
               +-----------------------+
                           |
               if rounds_since_todo >= 3:
                 inject <reminder> into tool_result
```

工作原理

1. TodoManager 存储带状态的项目。同一时间只允许一个 `in_progress`。

```
class TodoManager:
    def update(self, items: list) -> str:
        validated, in_progress_count = [], 0
        for item in items:
            status = item.get("status", "pending")
            if status == "in_progress":
                in_progress_count += 1
            validated.append({"id": item["id"], "text": item["
                                "status": status})
        if in_progress_count > 1:
            raise ValueError("Only one task can be in_progress
        self.items = validated
        return self.render()
```

1. `todo` 工具和其他工具一样加入 dispatch map。

```
TOOL_HANDLERS = {
    # ...base tools...
    "todo": lambda **kw: TODO.update(kw["items"]),
}
```

1. nag reminder: 模型连续 3 轮以上不调用 `todo` 时注入提醒。

```
if rounds_since_todo >= 3 and messages:
    last = messages[-1]
```

```
    if last["role"] == "user" and isinstance(last.get("content
        last["content"].insert(0, {
            "type": "text",
            "text": "<reminder>Update your todos.</reminder>",
        })
```

"同时只能有一个 in_progress" 强制顺序聚焦。nag reminder 制造问责压力 -- 你不更新计划, 系统就追着你问。

相对 s02 的变更

组件	之前 (s02)	之后 (s03)
Tools	4	5 (+todo)
规划	无	带状态的 TodoManager
Nag 注入	无	3 轮后注入 `<reminder>`
Agent loop	简单分发	+ rounds_since_todo 计数器

试一试

```
cd learn-claude-code
python agents/s03_todo_write.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Refactor the file hello.py: add type hints, docstrings, and a main guard`
2. `Create a Python package with __init__.py, utils.py, and tests/test_utils.py`
3. `Review all Python files and fix any style issues`

---

s04: Subagents (子智能体)

`s01 > s02 > s03 > [ s04 ] s05 > s06 | s07 > s08 > s09 > s10 > s11 > s12`

> *"大任务拆小, 每个小任务干净的上下文"* -- 子智能体用独立 messages[], 不污染主对话。

问题

智能体工作越久, messages 数组越胖。每次读文件、跑命令的输出都永久留在上下文里。"这个项目用什么测试框架?" 可能要读 5 个文件, 但父智能体只需要一个词: "pytest。"

解决方案

```
Parent agent                      Subagent
+------------------+              +------------------+
| messages=[...]   |              | messages=[]      | <-- fres
|                  |  dispatch    |                  |
| tool: task       | ----------> | while tool_use:  |
|   prompt="..."   |              |   call tools     |
|                  |  summary     |   append results |
|   result = "..." | <---------- | return last text |
+------------------+              +------------------+

Parent context stays clean. Subagent context is discarded.
```

工作原理

1. 父智能体有一个 `task` 工具。子智能体拥有除 `task` 外的所有基础工具 (禁止递归生成)。

```
PARENT_TOOLS = CHILD_TOOLS + [
    {"name": "task",
     "description": "Spawn a subagent with fresh context.",
     "input_schema": {
         "type": "object",
         "properties": {"prompt": {"type": "string"}},
         "required": ["prompt"],
     }},
]
```

1. 子智能体以 `messages=[]` 启动, 运行自己的循环。只有最终文本返回给父智能体。

```
def run_subagent(prompt: str) -> str:
    sub_messages = [{"role": "user", "content": prompt}]
    for _ in range(30):  # safety limit
        response = client.messages.create(
            model=MODEL, system=SUBAGENT_SYSTEM,
            messages=sub_messages,
            tools=CHILD_TOOLS, max_tokens=8000,
        )
        sub_messages.append({"role": "assistant",
                             "content": response.content})
        if response.stop_reason != "tool_use":
            break
        results = []
        for block in response.content:
            if block.type == "tool_use":
                handler = TOOL_HANDLERS.get(block.name)
                output = handler(**block.input)
                results.append({"type": "tool_result",
                    "tool_use_id": block.id,
                    "content": str(output)[:50000]})
        sub_messages.append({"role": "user", "content": result
    return "".join(
```

```
        b.text for b in response.content if hasattr(b, "text")
    ) or "(no summary)"
```

子智能体可能跑了 30+ 次工具调用, 但整个消息历史直接丢弃。父智能体收到的只是一段摘要文本, 作为普通 `tool_result` 返回。

相对 s03 的变更

组件	之前 (s03)	之后 (s04)
Tools	5	5 (基础) + task (仅父端)
上下文	单一共享	父 + 子隔离
Subagent	无	`run_subagent()` 函数
返回值	不适用	仅摘要文本

试一试

```
cd learn-claude-code
python agents/s04_subagent.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Use a subtask to find what testing framework this project uses`
2. `Delegate: read all .py files and summarize what each one does`
3. `Use a task to create a new module, then verify it from here`

---

s05: Skills (技能加载)

`s01 > s02 > s03 > s04 > [ s05 ] s06 | s07 > s08 > s09 > s10 > s11 > s12`

> *"用到什么知识, 临时加载什么知识"* -- 通过 tool_result 注入, 不塞 system prompt。

问题

你希望智能体遵循特定领域的工作流: git 约定、测试模式、代码审查清单。全塞进系统提示太浪费 -- 10 个技能, 每个 2000 token, 就是 20,000 token, 大部分跟当前任务毫无关系。

解决方案

```
System prompt (Layer 1 -- always present):
+--------------------------------------+
| You are a coding agent.              |
| Skills available:                    |
|   - git: Git workflow helpers        |  ~100 tokens/skill
|   - test: Testing best practices     |
+--------------------------------------+

When model calls load_skill("git"):
+--------------------------------------+
| tool_result (Layer 2 -- on demand):  |
| <skill name="git">                   |
|   Full git workflow instructions...  |  ~2000 tokens
|   Step 1: ...                        |
| </skill>                             |
+--------------------------------------+
```

第一层: 系统提示中放技能名称 (低成本)。第二层: tool_result 中按需放完整内容。

工作原理

1. 每个技能是一个目录, 包含 `SKILL.md` 文件和 YAML frontmatter。

```
skills/
  pdf/
    SKILL.md       # ---\n name: pdf\n description: Process PD
  code-review/
    SKILL.md       # ---\n name: code-review\n description: Re
```

1. SkillLoader 递归扫描 `SKILL.md` 文件, 用目录名作为技能标识。

```
class SkillLoader:
    def __init__(self, skills_dir: Path):
        self.skills = {}
        for f in sorted(skills_dir.rglob("SKILL.md")):
            text = f.read_text()
            meta, body = self._parse_frontmatter(text)
            name = meta.get("name", f.parent.name)
            self.skills[name] = {"meta": meta, "body": body}

    def get_descriptions(self) -> str:
        lines = []
        for name, skill in self.skills.items():
            desc = skill["meta"].get("description", "")
            lines.append(f"  - {name}: {desc}")
        return "\n".join(lines)

    def get_content(self, name: str) -> str:
        skill = self.skills.get(name)
        if not skill:
            return f"Error: Unknown skill '{name}'."
        return f"<skill name=\"{name}\">\n{skill['body']}\n</s
```

1. 第一层写入系统提示。第二层不过是 dispatch map 中的又一个工具。

```
SYSTEM = f"""You are a coding agent at {WORKDIR}.
Skills available:
```

```
{SKILL_LOADER.get_descriptions()}"""

TOOL_HANDLERS = {
    # ...base tools...
    "load_skill": lambda **kw: SKILL_LOADER.get_content(kw["na
}
```

模型知道有哪些技能 (便宜), 需要时再加载完整内容 (贵)。

相对 s04 的变更

组件	之前 (s04)	之后 (s05)
Tools	5 (基础 + task)	5 (基础 + load_skill)
系统提示	静态字符串	+ 技能描述列表
知识库	无	skills/*/SKILL.md 文件
注入方式	无	两层 (系统提示 + result)

试一试

```
cd learn-claude-code
python agents/s05_skill_loading.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `What skills are available?`
2. `Load the agent-builder skill and follow its instructions`
3. `I need to do a code review -- load the relevant skill first`
4. `Build an MCP server using the mcp-builder skill`

---

s06: Context Compact (上下文压缩)

`s01 > s02 > s03 > s04 > s05 > [ s06 ] | s07 > s08 > s09 > s10 > s11 > s12`

> *"上下文总会满, 要有办法腾地方"* -- 三层压缩策略, 换来无限会话。

问题

上下文窗口是有限的。读一个 1000 行的文件就吃掉 ~4000 token; 读 30 个文件、跑 20 条命令, 轻松突破 100k token。不压缩, 智能体根本没法在大项目里干活。

解决方案

三层压缩, 激进程度递增:

```
Every turn:
+------------------+
| Tool call result |
+------------------+
        |
        v
[Layer 1: micro_compact]        (silent, every turn)
  Replace tool_result > 3 turns old
  with "[Previous: used {tool_name}]"
        |
        v
[Check: tokens > 50000?]
   |               |
   no              yes
   |               |
   v               v
continue    [Layer 2: auto_compact]
              Save transcript to .transcripts/
              LLM summarizes conversation.
```

```
              Replace all messages with [summary].
                    |
                    v
            [Layer 3: compact tool]
              Model calls compact explicitly.
              Same summarization as auto_compact.
```

工作原理

1. **第一层 -- micro_compact**: 每次 LLM 调用前, 将旧的 tool result 替换为占位符。

```
def micro_compact(messages: list) -> list:
    tool_results = []
    for i, msg in enumerate(messages):
        if msg["role"] == "user" and isinstance(msg.get("conte
            for j, part in enumerate(msg["content"]):
                if isinstance(part, dict) and part.get("type")
                    tool_results.append((i, j, part))
    if len(tool_results) <= KEEP_RECENT:
        return messages
    for _, _, part in tool_results[:-KEEP_RECENT]:
        if len(part.get("content", "")) > 100:
            part["content"] = f"[Previous: used {tool_name}]"
    return messages
```

1. **第二层 -- auto_compact**: token 超过阈值时, 保存完整对话到磁盘, 让 LLM 做摘要。

```
def auto_compact(messages: list) -> list:
    # Save transcript for recovery
    transcript_path = TRANSCRIPT_DIR / f"transcript_{int(time.
    with open(transcript_path, "w") as f:
        for msg in messages:
            f.write(json.dumps(msg, default=str) + "\n")
    # LLM summarizes
```

```
    response = client.messages.create(
        model=MODEL,
        messages=[{"role": "user", "content":
            "Summarize this conversation for continuity..."
            + json.dumps(messages, default=str)[:80000]}],
        max_tokens=2000,
    )
    return [
        {"role": "user", "content": f"[Compressed]\n\n{respons
        {"role": "assistant", "content": "Understood. Continui
    ]
```

1. **第三层 -- manual compact**: `compact` 工具按需触发同样的摘要机制。
2. 循环整合三层:

```
def agent_loop(messages: list):
    while True:
        micro_compact(messages)                         # Layer
        if estimate_tokens(messages) > THRESHOLD:
            messages[:] = auto_compact(messages)        # Layer
        response = client.messages.create(...)
        # ... tool execution ...
        if manual_compact:
            messages[:] = auto_compact(messages)        # Layer
```

完整历史通过 transcript 保存在磁盘上。信息没有真正丢失, 只是移出了活跃上下文。

相对 s05 的变更

组件	之前 (s05)	之后 (s06)
Tools	5	5 (基础 + compact)
上下文管理	无	三层压缩
Micro-compact	无	旧结果 -> 占位符

组件	之前 (s05)	之后 (s06)
Auto-compact	无	token 阈值触发
Transcripts	无	保存到 .transcripts/

试一试

```
cd learn-claude-code
python agents/s06_context_compact.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Read every Python file in the agents/ directory one by one` (观察 micro-compact 替换旧结果)
2. `Keep reading files until compression triggers automatically`
3. `Use the compact tool to manually compress the conversation`

---

s07: Task System (任务系统)

`s01 > s02 > s03 > s04 > s05 > s06 | [ s07 ] s08 > s09 > s10 > s11 > s12`

> *"大目标要拆成小任务, 排好序, 记在磁盘上"* -- 文件持久化的任务图, 为多 agent 协作打基础。

问题

s03 的 TodoManager 只是内存中的扁平清单: 没有顺序、没有依赖、状态只有做完没做完。真实目标是有结构的 -- 任务 B 依赖任务 A, 任务 C 和 D 可以并行, 任务 E 要等 C 和 D 都完成。

没有显式的关系, 智能体分不清什么能做、什么被卡住、什么能同时跑。而且清单只活在内存里, 上下文压缩 (s06) 一跑就没了。

解决方案

把扁平清单升级为持久化到磁盘的**任务图**。每个任务是一个 JSON 文件, 有状态、前置依赖 (`blockedBy`) 和后置依赖 (`blocks`)。任务图随时回答三个问题:

- **什么可以做?** -- 状态为 `pending` 且 `blockedBy` 为空的任务。
- **什么被卡住?** -- 等待前置任务完成的任务。
- **什么做完了?** -- 状态为 `completed` 的任务, 完成时自动解锁后续任务。

```
.tasks/
  task_1.json  {"id":1, "status":"completed"}
  task_2.json  {"id":2, "blockedBy":[1], "status":"pending"}
  task_3.json  {"id":3, "blockedBy":[1], "status":"pending"}
  task_4.json  {"id":4, "blockedBy":[2,3], "status":"pending"}

任务图 (DAG):
                  +----------+
            +--> | task 2   | --+
            |    | pending  |   |
+----------+     +----------+    +--> +----------+
| task 1   |                         | task 4   |
| completed| --> +----------+    +--> | blocked  |
+----------+     | task 3   | --+     +----------+
                 | pending  |
                 +----------+

顺序:   task 1 必须先完成, 才能开始 2 和 3
并行:   task 2 和 3 可以同时执行
依赖:   task 4 要等 2 和 3 都完成
状态:   pending -> in_progress -> completed
```

这个任务图是 s07 之后所有机制的协调骨架: 后台执行 (s08)、多 agent 团队 (s09+)、worktree 隔离 (s12) 都读写这同一个结构。

工作原理

1. **TaskManager**: 每个任务一个 JSON 文件, CRUD + 依赖图。

```
class TaskManager:
    def __init__(self, tasks_dir: Path):
        self.dir = tasks_dir
        self.dir.mkdir(exist_ok=True)
        self._next_id = self._max_id() + 1

    def create(self, subject, description=""):
        task = {"id": self._next_id, "subject": subject,
                "status": "pending", "blockedBy": [],
                "blocks": [], "owner": ""}
        self._save(task)
        self._next_id += 1
        return json.dumps(task, indent=2)
```

1. **依赖解除**: 完成任务时, 自动将其 ID 从其他任务的 `blockedBy` 中移除, 解锁后续任务。

```
def _clear_dependency(self, completed_id):
    for f in self.dir.glob("task_*.json"):
        task = json.loads(f.read_text())
        if completed_id in task.get("blockedBy", []):
            task["blockedBy"].remove(completed_id)
            self._save(task)
```

1. **状态变更 + 依赖关联**: `update` 处理状态转换和依赖边。

```
def update(self, task_id, status=None,
           add_blocked_by=None, add_blocks=None):
    task = self._load(task_id)
    if status:
        task["status"] = status
        if status == "completed":
```

```
            self._clear_dependency(task_id)
    self._save(task)
```

1. 四个任务工具加入 dispatch map。

```
TOOL_HANDLERS = {
    # ...base tools...
    "task_create": lambda **kw: TASKS.create(kw["subject"]),
    "task_update": lambda **kw: TASKS.update(kw["task_id"], kw
    "task_list":   lambda **kw: TASKS.list_all(),
    "task_get":    lambda **kw: TASKS.get(kw["task_id"]),
}
```

从 s07 起, 任务图是多步工作的默认选择。s03 的 Todo 仍可用于单次会话内的快速清单。

相对 s06 的变更

组件	之前 (s06)	之后 (s07)
Tools	5	8 (`task_create/update/list/get`)
规划模型	扁平清单 (仅内存)	带依赖关系的任务图 (磁盘)
关系	无	`blockedBy` + `blocks` 边
状态追踪	做完没做完	`pending` -> `in_progress` -> `completed`
持久化	压缩后丢失	压缩和重启后存活

试一试

```
cd learn-claude-code
python agents/s07_task_system.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Create 3 tasks: "Setup project", "Write code", "Write tests". Make them depend on each other in order.`
2. `List all tasks and show the dependency graph`
3. `Complete task 1 and then list tasks to see task 2 unblocked`
4. `Create a task board for refactoring: parse -> transform -> emit -> test, where transform and emit can run in parallel after parse`

---

s08: Background Tasks (后台任务)

`s01 > s02 > s03 > s04 > s05 > s06 | s07 > [ s08 ] s09 > s10 > s11 > s12`

> *"慢操作丢后台, agent 继续想下一步"* -- 后台线程跑命令, 完成后注入通知。

问题

有些命令要跑好几分钟: `npm install`、`pytest`、`docker build`。阻塞式循环下模型只能干等。用户说 "装依赖, 顺便建个配置文件", 智能体却只能一个一个来。

解决方案

```
Main thread                     Background thread
+-----------------+             +-----------------+
| agent loop      |             | subprocess runs |
| ...             |             | ...             |
| [LLM call] <---+------- | enqueue(result) |
|  ^drain queue   |             +-----------------+
+-----------------+

Timeline:
Agent --[spawn A]--[spawn B]--[other work]----
             |           |
```

```
            v          v
         [A runs]   [B runs]      (parallel)
            |          |
            +-- results injected before next LLM call --+
```

工作原理

1. BackgroundManager 用线程安全的通知队列追踪任务。

```
class BackgroundManager:
    def __init__(self):
        self.tasks = {}
        self._notification_queue = []
        self._lock = threading.Lock()
```

1. `run()` 启动守护线程, 立即返回。

```
def run(self, command: str) -> str:
    task_id = str(uuid.uuid4())[:8]
    self.tasks[task_id] = {"status": "running", "command": com
    thread = threading.Thread(
        target=self._execute, args=(task_id, command), daemon=
    thread.start()
    return f"Background task {task_id} started"
```

1. 子进程完成后, 结果进入通知队列。

```
def _execute(self, task_id, command):
    try:
        r = subprocess.run(command, shell=True, cwd=WORKDIR,
            capture_output=True, text=True, timeout=300)
        output = (r.stdout + r.stderr).strip()[:50000]
    except subprocess.TimeoutExpired:
        output = "Error: Timeout (300s)"
    with self._lock:
```

```
        self._notification_queue.append({
            "task_id": task_id, "result": output[:500]})
```

1. 每次 LLM 调用前排空通知队列。

```
def agent_loop(messages: list):
    while True:
        notifs = BG.drain_notifications()
        if notifs:
            notif_text = "\n".join(
                f"[bg:{n['task_id']}] {n['result']}" for n in
            messages.append({"role": "user",
                "content": f"<background-results>\n{notif_text
                           f"</background-results>"})
            messages.append({"role": "assistant",
                "content": "Noted background results."})
        response = client.messages.create(...)
```

循环保持单线程。只有子进程 I/O 被并行化。

相对 s07 的变更

组件	之前 (s07)	之后 (s08)
Tools	8	6 (基础 + background_run + check)
执行方式	仅阻塞	阻塞 + 后台线程
通知机制	无	每轮排空的队列
并发	无	守护线程

试一试

```
cd learn-claude-code
```

```
python agents/s08_background_tasks.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Run "sleep 5 && echo done" in the background, then create a file while it runs`
2. `Start 3 background tasks: "sleep 2", "sleep 4", "sleep 6". Check their status.`
3. `Run pytest in the background and keep working on other things`

---

s09: Agent Teams (智能体团队)

`s01 > s02 > s03 > s04 > s05 > s06 | s07 > s08 > [ s09 ] s10 > s11 > s12`

> *"任务太大一个人干不完, 要能分给队友"* -- 持久化队友 + JSONL 邮箱。

问题

子智能体 (s04) 是一次性的: 生成、干活、返回摘要、消亡。没有身份, 没有跨调用的记忆。后台任务 (s08) 能跑 shell 命令, 但做不了 LLM 引导的决策。

真正的团队协作需要三样东西: (1) 能跨多轮对话存活的持久智能体, (2) 身份和生命周期管理, (3) 智能体之间的通信通道。

解决方案

```
Teammate lifecycle:
  spawn -> WORKING -> IDLE -> WORKING -> ... -> SHUTDOWN

Communication:
  .team/
    config.json           <- team roster + statuses
```

```
    inbox/
      alice.jsonl         <- append-only, drain-on-read
      bob.jsonl
      lead.jsonl

              +--------+    send("alice","bob","...")    +----
              | alice  | ----------------------------> |  bob
              | loop   |    bob.jsonl << {json_line}    |  loo
              +--------+                                +-----
                   ^                                         |
                   |        BUS.read_inbox("alice")          |
                   +---- alice.jsonl -> read + drain ---------
```

工作原理

1. TeammateManager 通过 config.json 维护团队名册。

```
class TeammateManager:
    def __init__(self, team_dir: Path):
        self.dir = team_dir
        self.dir.mkdir(exist_ok=True)
        self.config_path = self.dir / "config.json"
        self.config = self._load_config()
        self.threads = {}
```

1. `spawn()` 创建队友并在线程中启动 agent loop。

```
def spawn(self, name: str, role: str, prompt: str) -> str:
    member = {"name": name, "role": role, "status": "working"}
    self.config["members"].append(member)
    self._save_config()
    thread = threading.Thread(
        target=self._teammate_loop,
        args=(name, role, prompt), daemon=True)
```

```
    thread.start()
    return f"Spawned teammate '{name}' (role: {role})"
```

1. MessageBus: append-only 的 JSONL 收件箱。`send()` 追加一行; `read_inbox()` 读取全部并清空。

```
class MessageBus:
    def send(self, sender, to, content, msg_type="message", ex
        msg = {"type": msg_type, "from": sender,
               "content": content, "timestamp": time.time()}
        if extra:
            msg.update(extra)
        with open(self.dir / f"{to}.jsonl", "a") as f:
            f.write(json.dumps(msg) + "\n")

    def read_inbox(self, name):
        path = self.dir / f"{name}.jsonl"
        if not path.exists(): return "[]"
        msgs = [json.loads(l) for l in path.read_text().strip(
        path.write_text("")  # drain
        return json.dumps(msgs, indent=2)
```

1. 每个队友在每次 LLM 调用前检查收件箱, 将消息注入上下文。

```
def _teammate_loop(self, name, role, prompt):
    messages = [{"role": "user", "content": prompt}]
    for _ in range(50):
        inbox = BUS.read_inbox(name)
        if inbox != "[]":
            messages.append({"role": "user",
                "content": f"<inbox>{inbox}</inbox>"})
            messages.append({"role": "assistant",
                "content": "Noted inbox messages."})
        response = client.messages.create(...)
        if response.stop_reason != "tool_use":
            break
```

```
        # execute tools, append results...
    self._find_member(name)["status"] = "idle"
```

相对 s08 的变更

组件	之前 (s08)	之后 (s09)
Tools	6	9 (+spawn/send/read_inbox)
智能体数量	单一	领导 + N 个队友
持久化	无	config.json + JSONL 收件箱
线程	后台命令	每线程完整 agent loop
生命周期	一次性	idle -> working -> idle
通信	无	message + broadcast

试一试

```
cd learn-claude-code
python agents/s09_agent_teams.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Spawn alice (coder) and bob (tester). Have alice send bob a message.`
2. `Broadcast "status update: phase 1 complete" to all teammates`
3. `Check the lead inbox for any messages`
4. 输入 `/team` 查看团队名册和状态
5. 输入 `/inbox` 手动检查领导的收件箱

---

s10: Team Protocols (团队协议)

`s01 > s02 > s03 > s04 > s05 > s06 | s07 > s08 > s09 > [ s10 ] s11 > s12`

> *"队友之间要有统一的沟通规矩"* -- 一个 request-response 模式驱动所有协商。

问题

s09 中队友能干活能通信, 但缺少结构化协调:

关机: 直接杀线程会留下写了一半的文件和过期的 config.json。需要握手 -- 领导请求, 队友批准 (收尾退出) 或拒绝 (继续干)。

计划审批: 领导说 "重构认证模块", 队友立刻开干。高风险变更应该先过审。

两者结构一样: 一方发带唯一 ID 的请求, 另一方引用同一 ID 响应。

解决方案

```
Shutdown Protocol            Plan Approval Protocol
==================           ======================

Lead              Teammate    Teammate           Lead
  |                  |           |                  |
  |--shutdown_req-->|           |--plan_req------>|
  | {req_id:"abc"}  |           | {req_id:"xyz"}  |
  |                  |           |                  |
  |<--shutdown_resp-|           |<--plan_resp-----|
  | {req_id:"abc",  |           | {req_id:"xyz",  |
  |  approve:true}  |           |  approve:true}  |

Shared FSM:
  [pending] --approve--> [approved]
  [pending] --reject---> [rejected]
```

```
Trackers:
  shutdown_requests = {req_id: {target, status}}
  plan_requests     = {req_id: {from, plan, status}}
```

工作原理

1. 领导生成 request_id, 通过收件箱发起关机请求。

```
shutdown_requests = {}

def handle_shutdown_request(teammate: str) -> str:
    req_id = str(uuid.uuid4())[:8]
    shutdown_requests[req_id] = {"target": teammate, "status":
    BUS.send("lead", teammate, "Please shut down gracefully.",
             "shutdown_request", {"request_id": req_id})
    return f"Shutdown request {req_id} sent (status: pending)"
```

1. 队友收到请求后, 用 approve/reject 响应。

```
if tool_name == "shutdown_response":
    req_id = args["request_id"]
    approve = args["approve"]
    shutdown_requests[req_id]["status"] = "approved" if approv
    BUS.send(sender, "lead", args.get("reason", ""),
             "shutdown_response",
             {"request_id": req_id, "approve": approve})
```

1. 计划审批遵循完全相同的模式。队友提交计划 (生成 request_id), 领导审查 (引用同一个 request_id)。

```
plan_requests = {}

def handle_plan_review(request_id, approve, feedback=""):
    req = plan_requests[request_id]
    req["status"] = "approved" if approve else "rejected"
```

```
    BUS.send("lead", req["from"], feedback,
             "plan_approval_response",
             {"request_id": request_id, "approve": approve})
```

一个 FSM, 两种用途。同样的 `pending -> approved | rejected` 状态机可以套用到任何请求-响应协议上。

相对 s09 的变更

组件	之前 (s09)	之后 (s10)
Tools	9	12 (+shutdown_req/resp +plan)
关机	仅自然退出	请求-响应握手
计划门控	无	提交/审查与审批
关联	无	每个请求一个 request_id
FSM	无	pending -> approved/rejected

试一试

```
cd learn-claude-code
python agents/s10_team_protocols.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Spawn alice as a coder. Then request her shutdown.`
2. `List teammates to see alice's status after shutdown approval`
3. `Spawn bob with a risky refactoring task. Review and reject his plan.`
4. `Spawn charlie, have him submit a plan, then approve it.`
5. 输入 `/team` 监控状态

---

s11: Autonomous Agents (自治智能体)

`s01 > s02 > s03 > s04 > s05 > s06 | s07 > s08 > s09 > s10 > [ s11 ] s12`

> *"队友自己看看板, 有活就认领"* -- 不需要领导逐个分配, 自组织。

问题

s09-s10 中, 队友只在被明确指派时才动。领导得给每个队友写 prompt, 任务看板上 10 个未认领的任务得手动分配。这扩展不了。

真正的自治: 队友自己扫描任务看板, 认领没人做的任务, 做完再找下一个。

一个细节: 上下文压缩 (s06) 后智能体可能忘了自己是谁。身份重注入解决这个问题。

解决方案

```
Teammate lifecycle with idle cycle:

+-------+
| spawn |
+---+---+
    |
    v
+-------+   tool_use     +-------+
| WORK  | <------------- |  LLM  |
+---+---+                +-------+
    |
    | stop_reason != tool_use (or idle tool called)
    v
+--------+
|  IDLE  |  poll every 5s for up to 60s
+---+----+
    |
```

```
    +---> check inbox --> message? ----------> WORK
    |
    +---> scan .tasks/ --> unclaimed? -------> claim -> WORK
    |
    +---> 60s timeout ----------------------> SHUTDOWN

Identity re-injection after compression:
  if len(messages) <= 3:
    messages.insert(0, identity_block)
```

工作原理

1. 队友循环分两个阶段: WORK 和 IDLE。LLM 停止调用工具 (或调用了 `idle`) 时, 进入 IDLE。

```
def _loop(self, name, role, prompt):
    while True:
        # -- WORK PHASE --
        messages = [{"role": "user", "content": prompt}]
        for _ in range(50):
            response = client.messages.create(...)
            if response.stop_reason != "tool_use":
                break
            # execute tools...
            if idle_requested:
                break

        # -- IDLE PHASE --
        self._set_status(name, "idle")
        resume = self._idle_poll(name, messages)
        if not resume:
            self._set_status(name, "shutdown")
            return
        self._set_status(name, "working")
```

1. 空闲阶段循环轮询收件箱和任务看板。

```
def _idle_poll(self, name, messages):
    for _ in range(IDLE_TIMEOUT // POLL_INTERVAL):  # 60s / 5s
        time.sleep(POLL_INTERVAL)
        inbox = BUS.read_inbox(name)
        if inbox:
            messages.append({"role": "user",
                "content": f"<inbox>{inbox}</inbox>"})
            return True
        unclaimed = scan_unclaimed_tasks()
        if unclaimed:
            claim_task(unclaimed[0]["id"], name)
            messages.append({"role": "user",
                "content": f"<auto-claimed>Task #{unclaimed[0]
                           f"{unclaimed[0]['subject']}</auto-c
            return True
    return False  # timeout -> shutdown
```

1. 任务看板扫描: 找 pending 状态、无 owner、未被阻塞的任务。

```
def scan_unclaimed_tasks() -> list:
    unclaimed = []
    for f in sorted(TASKS_DIR.glob("task_*.json")):
        task = json.loads(f.read_text())
        if (task.get("status") == "pending"
                and not task.get("owner")
                and not task.get("blockedBy")):
            unclaimed.append(task)
    return unclaimed
```

1. 身份重注入: 上下文过短 (说明发生了压缩) 时, 在开头插入身份块。

```
if len(messages) <= 3:
    messages.insert(0, {"role": "user",
        "content": f"<identity>You are '{name}', role: {role},
                   f"team: {team_name}. Continue your work.</i
```

```
    messages.insert(1, {"role": "assistant",
        "content": f"I am {name}. Continuing."})
```

相对 s10 的变更

组件	之前 (s10)	之后 (s11)
Tools	12	14 (+idle, +claim_task)
自治性	领导指派	自组织
空闲阶段	无	轮询收件箱 + 任务看板
任务认领	仅手动	自动认领未分配任务
身份	系统提示	+ 压缩后重注入
超时	无	60 秒空闲 -> 自动关机

试一试

```
cd learn-claude-code
python agents/s11_autonomous_agents.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Create 3 tasks on the board, then spawn alice and bob. Watch them auto-claim.`
2. `Spawn a coder teammate and let it find work from the task board itself`
3. `Create tasks with dependencies. Watch teammates respect the blocked order.`
4. 输入 `/tasks` 查看带 owner 的任务看板
5. 输入 `/team` 监控谁在工作、谁在空闲

---

s12: Worktree + Task Isolation (Worktree 任务隔离)

`s01 > s02 > s03 > s04 > s05 > s06 | s07 > s08 > s09 > s10 > s11 > [ s12 ]`

> *"各干各的目录, 互不干扰"* -- 任务管目标, worktree 管目录, 按 ID 绑定。

问题

到 s11, 智能体已经能自主认领和完成任务。但所有任务共享一个目录。两个智能体同时重构不同模块 -- A 改 `config.py`, B 也改 `config.py`, 未提交的改动互相污染, 谁也没法干净回滚。

任务板管 "做什么" 但不管 "在哪做"。解法: 给每个任务一个独立的 git worktree 目录, 用任务 ID 把两边关联起来。

解决方案

```
Control plane (.tasks/)             Execution plane (.worktree
+------------------+                +------------------------+
| task_1.json      |                | auth-refactor/         |
|   status: in_progress  <------>   branch: wt/auth-refactor
|   worktree: "auth-refactor"   |   task_id: 1             |
+------------------+                +------------------------+
| task_2.json      |                | ui-login/              |
|   status: pending    <------>     branch: wt/ui-login
|   worktree: "ui-login"       |   task_id: 2             |
+------------------+                +------------------------+
                                    |
                           index.json (worktree registry)
                           events.jsonl (lifecycle log)


State machines:
```

```
  Task:     pending -> in_progress -> completed
  Worktree: absent  -> active      -> removed | kept
```

工作原理

1. **创建任务。** 先把目标持久化。

```
TASKS.create("Implement auth refactor")
# -> .tasks/task_1.json  status=pending  worktree=""
```

1. **创建 worktree 并绑定任务。** 传入 `task_id` 自动将任务推进到 `in_progress` 。

```
WORKTREES.create("auth-refactor", task_id=1)
# -> git worktree add -b wt/auth-refactor .worktrees/auth-refa
# -> index.json gets new entry, task_1.json gets worktree="aut
```

绑定同时写入两侧状态:

```
def bind_worktree(self, task_id, worktree):
    task = self._load(task_id)
    task["worktree"] = worktree
    if task["status"] == "pending":
        task["status"] = "in_progress"
    self._save(task)
```

1. **在 worktree 中执行命令。** `cwd` 指向隔离目录。

```
subprocess.run(command, shell=True, cwd=worktree_path,
               capture_output=True, text=True, timeout=300)
```

1. **收尾。** 两种选择:
2. `worktree_keep(name)` -- 保留目录供后续使用。

3. `worktree_remove(name, complete_task=True)` -- 删除目录, 完成绑定任务, 发出事件。一个调用搞定拆除 + 完成。

```
def remove(self, name, force=False, complete_task=False):
    self._run_git(["worktree", "remove", wt["path"]])
    if complete_task and wt.get("task_id") is not None:
        self.tasks.update(wt["task_id"], status="completed")
        self.tasks.unbind_worktree(wt["task_id"])
        self.events.emit("task.completed", ...)
```

1. **事件流。** 每个生命周期步骤写入 `.worktrees/events.jsonl`:

```
{
  "event": "worktree.remove.after",
  "task": {"id": 1, "status": "completed"},
  "worktree": {"name": "auth-refactor", "status": "removed"},
  "ts": 1730000000
}
```

事件类型: `worktree.create.before/after/failed`, `worktree.remove.before/after/failed`, `worktree.keep`, `task.completed`。

崩溃后从 `.tasks/` + `.worktrees/index.json` 重建现场。会话记忆是易失的; 磁盘状态是持久的。

相对 s11 的变更

组件	之前 (s11)	之后 (s12)
协调	任务板 (owner/status)	任务板 + worktree 显式绑定
执行范围	共享目录	每个任务独立目录
可恢复性	仅任务状态	任务状态 + worktree 索引

组件	之前 (s11)	之后 (s12)
收尾	任务完成	任务完成 + 显式 keep/remove
生命周期可见性	隐式日志	`.worktrees/events.jsonl` 显式事件流

试一试

```
cd learn-claude-code
python agents/s12_worktree_task_isolation.py
```

试试这些 prompt (英文 prompt 对 LLM 效果更好, 也可以用中文):

1. `Create tasks for backend auth and frontend login page, then list tasks.`
2. `Create worktree "auth-refactor" for task 1, then bind task 2 to a new worktree "ui-login".`
3. `Run "git status --short" in worktree "auth-refactor".`
4. `Keep worktree "ui-login", then list worktrees and inspect events.`
5. `Remove worktree "auth-refactor" with complete_task=true, then list tasks/worktrees/events.`

总结

这 12 个章节展示了如何从一个简单的智能体循环逐步构建到完整的自治智能体团队系统:

章节	核心概念	新增机制
s01	Agent Loop	`while True` + `stop_reason`
s02	Tool Use	Dispatch map + 路径沙箱

章节	核心概念	新增机制
s03	TodoWrite	TodoManager + Nag reminder
s04	Subagents	上下文隔离的子智能体
s05	Skills	两层技能加载机制
s06	Context Compact	三层压缩策略
s07	Task System	持久化任务图 (DAG)
s08	Background Tasks	后台线程 + 通知队列
s09	Agent Teams	多智能体 + JSONL 邮箱
s10	Team Protocols	Request-Response 协议
s11	Autonomous Agents	自组织 + 空闲轮询
s12	Worktree Isolation	任务-目录绑定隔离

> **核心原则**: 循环本身始终不变，所有机制都是叠加在循环之上的增量改进。